

Lagersystem

Detaillierter Projektbericht

Vom Excel/VBA-Originalsystem über die lokale Streamlit-App zur privaten
Cloud-Anwendung

Ausführliche Fassung - inkl. vollständiger Code-Architektur des Originalsystems

Stand: 16. August 2026

Inhaltsuebersicht

Teil I - Das Original-Excel/VBA-System

1. Betriebsmodi: Tablet-Modus und Entwickler-Modus
2. Die Tabellenblaetter
3. Die Eingabemasken (UserForms)
4. Die Code-Module im Detail (A bis H)
5. Datenmodell der Excel-Tabellen
6. Erkannte Schwachstellen als Ausgangspunkt der Neuentwicklung

Teil II - Die Neuentwicklung als Web-Anwendung

7. Architekturentscheidung: Weg von Excel
8. Aufbau des Grundsystems
9. VBA-Funktion vs. Python-Funktion - eine Gegenueberstellung
10. Digitalisierung der Bestandsdaten
11. Iterative Weiterentwicklung der Oberflaeche
12. Datensicherheit: Papierkorb und Loesch-Bestaetigung

Teil III - Von lokal zur Cloud

13. Produktionsreife auf dem lokalen Rechner
14. Mehrgeraete-Zugriff und Windows-Vorbereitung
15. Die Entscheidung fuer die Cloud
16. Technische Umsetzung der Cloud-Migration
17. Go-Live: GitHub, Neon und Streamlit Community Cloud
18. Fehlerbehebung nach dem Go-Live
19. Aktueller Stand und offene Punkte

Teil I - Das Original-Excel/VBA-System

Bevor irgendetwas neu gebaut wurde, stand die vollstaendige Analyse des bestehenden Systems („Lager System V3.3.xlsm“). Dieses Kapitel dokumentiert dessen Aufbau im Detail, so wie er aus der Durchsicht aller Tabellenblaetter, UserForms und VBA-Module hervorgeht - als Grundlage dafuer, die fachliche Logik korrekt in die neue Anwendung zu uebertragen.

1. Betriebsmodi: Tablet-Modus und Entwickler-Modus

Die Excel-Datei kannte zwei Ansichts-Modi, umschaltbar per Button (Modul **A3_Tablet_Modus**, Sub APPModus_Umschalten): Im **Tablet-Modus** (App-Modus) werden Menueband (Ribbon), Bearbeitungsleiste, Spalten-/Zeilenkoepfe und die Blattregister-Reiter ausgeblendet und alle Blaetter automatisch auf ihren jeweiligen Datenbereich gezoomt (Modul **A4_AnsichtBereich**) - die Excel-Datei wirkt dadurch wie eine eigenstaendige App. Der **Entwickler-Modus** blendet alle diese Elemente wieder ein, um den Code zu bearbeiten. Bereits beim Oeffnen der Datei (workbook_Open in „DieseArbeitsmappe“) werden ausserdem alle Blaetter zunaechst entsperrt, bestimmte Zellbereiche gezielt fuer Eingaben freigegeben (z. B. nur Spalten A-G auf dem Blatt „Artikel Gruppe“, dagegen A-Q auf „Artikel Liste“) und anschliessend wieder komplett gesperrt (Blattschutz), damit Endnutzer nur ueber die vorgesehenen Buttons und Eingabefelder arbeiten koennen.

2. Die Tabellenblaetter

- **Dashboard** - zentrale Startseite mit Exportbereich (Blattauswahl, Excel/PDF, Zielordner, E-Mail-Empfaenger), einer Artikelsuche (Barcode oder Bezeichnung, Ergebnisse direkt im Dashboard eingeblendet) sowie Schnellzugriff-Buttons zu allen anderen Bereichen.
- **Artikel Liste** - alle vollen Artikel-Stuecke, mit Filter-/Entfilter-Buttons und den Aktionen Hinzufuegen/Loeschen/Bearbeiten.
- **Reste** - identisch aufgebaut wie Artikel Liste, fuer Reststuecke.
- **Verlauf** - reines, nicht editierbares Buchungsprotokoll (nur Filter/Entfilter, keine Hinzufuegen/Loeschen/Bearbeiten-Aktionen).
- **Artikel Gruppe** - Stammdaten je Artikelsorte (erzeugt den Haupt-Barcode); hier ist bewusst nur ein kleinerer Zellbereich (A-G) fuer Eingaben freigegeben.
- **Einstellung** - enthaelt die vier Nachschlage-Tabellen **Profil_Abk**, **Material_Gruppe**, **Lager** und **Einheit** (bzw. „Type“) nebeneinander auf einem Blatt.

3. Die Eingabemasken (UserForms)

frmArtikelDetails	Erfasst/bearbeitet ein einzelnes Artikel-Stueck: Laenge/mm, Ort/Regal (Dropdown aus Tabelle „Lager“), Type Voll/Rest (Dropdown aus „Type“), Menge/Stueck, E-Preis, E-Preis pro Einheit (Dropdown aus „Einheit“). Validiert Pflichtfelder, numerische Werte und negative Preise, bevor gespeichert werden darf.
frmArtikelGruppe	Legt eine neue Artikelgruppe an: Profil und Material je als Dropdown aus den Einstellungen-Tabellen, dazu ein freies Textfeld „Mass“. Alle drei Felder sind Pflicht.
frmBarcodeEingabe	Zentrale Barcode-Eingabe/-Suche: Barcode scannen oder eintippen, alternativ per Dropdown einen bestehenden Barcode oder eine Bezeichnung waehlen (beide Listen wechselseitig verknuepft - Auswahl der Bezeichnung traegt automatisch den passenden Barcode ein). Wird von mehreren anderen Funktionen wiederverwendet (Hinzufuegen, Loeschen, Bearbeiten, Materialentnahme).
frmMaterialEntnahme	Kernstueck der Entnahme-Logik: Barcode scannen, bei einem Haupt-Barcode erst ein konkretes Stueck aus einer automatisch befuellten Dropdown-Liste waehlen, dann die genutzte Laenge eintragen - die Restlaenge wird live waehrend der Eingabe berechnet (txtGenutzt_Change). Beim Bestaetigen prueft das Formular, ob die Restlaenge unter 1000 mm liegt, und fragt in diesem Fall per Ja/Nein-Dialog, ob das Stueck als Schrott entsorgt werden soll.

4. Die Code-Module im Detail (A bis H)

Der VBA-Code war in 24 Module gegliedert, alphabetisch nach acht Themenbereichen (A-H) sortiert:

A - Grundgeruest (Sicherheit, Navigation, Ansicht)

A1_Sicherheit	SheetsEntsperren / SheetsSperren (Blattschutz an/aus, mit hartkodiertem Passwort im Code), ZellenOeffnen (gibt gezielt einzelne Zellbereiche fuer Eingaben frei, bevor der Blattschutz greift).
A2_Navigation	Je eine Sub pro Zielblatt (ZuDashboard, ZuArtikelliste, ZuReste, ZuVerlauf, ZuArtikelGruppe, ZuEinstellungen) - hinter den Navigations-Buttons.
A3_Tablet_Modus	Umschalt-Logik zwischen App-Modus und Entwickler-Modus (siehe Kapitel 1).
A4_AnzichtBereich	ZoomAufSpalten - zoomt ein Blatt automatisch auf einen definierten Spaltenbereich, je Blatt mit individueller Spaltenbreite (z. B. „Artikel Gruppe“ nur A-K, „Dashboard“ A-U).

B - Export (Excel/PDF)

B1_Daten_Typen	Eigene VBA-Typen LayoutConfig (Papierformat/Ausrichtung) und ExportFormate (Excel/PDF je als Boolean).
B2_Exportieren_Hilfsfunktionen	Liest die Checkbox-Auswahl im Dashboard aus (welche Blaetter, welches Format, Papierlayout), fragt bei Bedarf per Ordnerdialog nach dem Zielordner - plattformabhaengig verzweigt : unter Mac per AppleScript-Ordnerauswahl, unter Windows per Application.FileDialog. Generiert automatisch einen Dateinamen aus Blattnamen und Zeitstempel.
B3_Exportieren	DashboardExport - die Sub hinter dem „Speichern“-Button, ruft die Hilfsfunktionen der Reihe nach auf und exportiert je nach Auswahl als PDF und/oder Excel.

C - E-Mail-Versand

C1_Datentypen	Eigener Typ EmailDaten (Empfaenger, Betreff, Text, zwei Dateianhaenge).
C4_Email_Hilfsfunktionen	Baut Betreff/Text automatisch zusammen, sucht die exportierte Datei wieder, und initialisiert den gewünschten Mail-Client. Bemerkenswert: Fuer Windows wird Outlook per COM angesteuert, fuer Mac Apple Mail per AppleScript - fuer Gmail existiert keine echte Automatisierung , der Code oeffnet im Zweifel nur das Programm-Fenster per Shell-Befehl und gibt ansonsten „Nothing“ zurueck.
C5_Email_senden	VerschickeExportPerEmail - fuehrt Export und E-Mail-Erstellung zusammen, mit erneuter Plattformweiche (#If Mac / #Else) fuer den eigentlichen Versand.

D - Universelle Hilfsfunktionen

D1_Univ_Barcode	HoleBarcodeEingabe (oeffnet frmBarcodeEingabe und liefert den eingegebenen Wert zurueck), FindeMaxBarcodeNummer (sucht die hoechste bereits vergebene Nummer zu einem Praefix - Basis der automatischen Barcode-Vergabe).
D2_In_Tabelle_suchen	Rowsuchen (durchsucht eine Tabelle spaltenweise nach einem Wert, zeigt bei Nichtfund automatisch eine Fehlermeldung), WertExistiertInTabelle (Ja/Nein-Pruefung, z. B. fuer Duplikat-Erkennung).
D3_Univ_Ent_Filtern	Generische Filter-Funktionen (TabelleFiltern/TabelleEntfiltern/DynamischFiltern) auf Basis von Excels AutoFilter, wiederverwendet fuer alle vier Filter-Buttons in Artikel Liste/Reste/Verlauf. Dazu Clear_Search_Results zum Leeren der Dashboard-Suchergebnisse.

E - Profil-/Material-Nachschlagewerke

E1_Profil_tabelle	HoleProfilAbkuerzung - schlaegt zu einem Profilnamen das Kuerzel fuer die Barcode-Bildung nach (Tabelle Profil_Abk).
E2_Material_tabelle	HoleMaterialAbkuerzung und HoleMaterialGruppe - analog fuer Material bzw. dessen uebergeordnete Materialgruppe.

F - Artikelgruppen-Verwaltung

F1_ArtikelGruppe_DatenType	Eigener Typ ArtikelGruppe mit den sechs Feldern Benone/Bez/Profil/Mas/Material/Gefunden.
F2_ArtGrp_Hilfsfunktionen	MakeNewData_AG (fragt per UserForm ab), MakeBarcode_AG (kombiniert Profil- und Material-Kuerzel plus naechste freie 4-stellige Nummer), InTabelleHinzufuegen_AG (schreibt die neue Zeile).
F3_ArtikelGruppe_Erstellen	MakeArtikelGruppe - die Sub hinter dem Button: fragt Daten ab, prueft auf Duplikate (WertExistiertInTabelle), generiert den Barcode, traegt ein und bestaetigt per MsgBox.
F4_Aus_ArtikelGruppe	HoleDaten_AG - laedt zu einem Barcode die vollstaendigen Stammdaten (Bezeichnung, Profil, Mass, Material) fuer die Weiterverarbeitung.

G - Artikel-Stueck-Verwaltung (Kernstueck des Systems)

G1_ArtikelStueck_Type	Typ ArtikelStueck mit 15 Feldern (Datum, Haupt-/Stueck-Barcode, Bezeichnung, Profil, Mass, Laenge, Material(-Gruppe), Ort, Typ, Menge, E-Preis, E-Preis-Einheit, Gefunden).
G2_ArtStk_Hilfsunktionen	MakeBarcode_AS (Stueck-Barcode = Haupt-Barcode + naechste freie laufende Nummer je Tabelle), InTabelleHinzufuegen_AS (schreibt eine Zeile - kann sowohl neue Zeilen anlegen als auch eine bestehende an einer bestimmten Position ueberschreiben, je nachdem ob ZeilenNummer 0 ist).
G3_ArtikelStueck_Hinzufuegen	MakeNew_ArtikelStueck - fragt Haupt-Barcode + Details per UserForm ab, laedt die Artikelgruppen-Stammdaten, generiert den Stueck-Barcode und traegt sowohl in die Zieltabelle als auch parallel ins Verlauf-Protokoll ein.
G4_ArtikelStueck_loeschen	Del_ArtikelStueck - fragt den Barcode ab (inkl. Nachfrage nach der Stuecknummer, falls nur der Haupt-Barcode gescannt wurde), zeigt eine Ja/Nein-Sicherheitsabfrage („WIRKLICH unwiderruflich loeschen?“) und entfernt die Zeile erst danach endgueltig.
G5_ArtikelStueck_Bearbeiten	edit_ArtikelStueck - laedt die bestehenden Werte in frmArtikelDetails vor, uebernimmt die Aenderungen und schreibt zusaetzlich einen Verlauf-Eintrag.
G6_ArtikelStueck_Buttons	Die konkreten Button-Makros je Tabelle (Artikel_hinzufuegen, Artikel_loechen [sic], Artikel_bearbeiten, Artikel_Filtern/Entfiltern und die Reste-Gegenstuecke), sowie MaterialEntnehmen_Artikel/MaterialEntnehmen_Reste, die frmMaterialEntnahme mit dem passenden Blatt-/Tabellenkontext oeffnen.
G7_MaterialEntnahme_Logik	Das fachliche Herzstueck: ProzessMaterialEntnahme koordiniert - in dieser Reihenfolge - den Verlauf-Eintrag (SchreibeEntnahmeinVerlauf, geschrieben <i>bevor</i> reduziert wird), die Reduktion/Loeschung des Ursprungsstuecks (ReduziereUrsprungsStueck: Menge -1, oder Zeile komplett loeschen bei Menge 1) und - falls kein Schrott - das Einbuchen des Reststuecks in die passende Zieltabelle (BucheRestStueckEin: sucht zunaechst nach einem identischen bestehenden Reststueck und erhoeht dessen Menge, legt sonst ein neues Stueck an). Enthaeft daneben FuelleComboBoxMitStuecken zum Befuellen der Stueck-Auswahlliste in frmMaterialEntnahme.

H - Dashboard-Suche

H1_Suchen_Funktionen	ValidateSearchInput und Fill_Dashboard_List. Bemerkenswert: ValidateSearchInput enthaelt an zentraler Stelle noch Platzhalter-Code (eine hartkodierte Test-Bezeichnung sowie mehrere auskommentierte Zeilen fuer einen geplanten, aber nie fertig umgesetzten „Mismatch“-Eingabedialog) - ein sichtbarer Hinweis darauf, dass die Suchfunktion im Originalsystem noch nicht vollstaendig ausimplementiert war.
H2_Suchbotton	BtnSuche_Click - liest die beiden Sucheingaben aus dem Dashboard, validiert sie und fuellt sowohl die Artikel-Liste- als auch die Reste-Trefferliste inklusive Summenbildung direkt in vordefinierte Dashboard-Zellen.

5. Datenmodell der Excel-Tabellen

Die drei zentralen Tabellen **Artikel_Liste**, **Reste** und **Verlauf** teilen sich denselben 14-spaltigen Aufbau (aus G1_ArtikelStueck_Type und den InTabelleHinzufuegen_AS-Zuweisungen rekonstruiert):

#	Spalte
1	Datum
2	Barcode (Haupt)
3	Barcode2 (Stueck)
4	Bezeichnung
5	Profil

6	Mass
7	Laenge/mm
8	Material-Gruppe
9	Material
10	Ort/Regal
11	Type (Voll/Rest)
12	Menge/Stueck
13	E-Preis
14	E-Preis pro Einheit

Die Tabelle **Artikel_Gruppe** ist mit fuef Spalten deutlich schmaeler: Barcode/Nummer, Bezeichnung, Profil, Mass, Material. Die Einstellungen-Tabellen **Profil_Abk** und **Material_Gruppe** bilden jeweils Name -> Kuerzel (Material_Gruppe zusaetzlich mit einer dritten Spalte fuer die uebergeordnete Materialgruppe) ab, **Lager** und **Einheit** sind einspaltige Listen mit Regalorten bzw. Mengeneinheiten.

6. Erkannte Schwachstellen als Ausgangspunkt der Neuentwicklung

Die detaillierte Durchsicht des Codes machte mehrere strukturelle Schwachpunkte sichtbar, die die Entscheidung fuer eine Neuentwicklung zusaetzlich untermauerten:

- **Kein Mehrbenutzerbetrieb:** Eine Excel-Datei kann von mehreren Personen nur eingeschaenkt gleichzeitig bearbeitet werden - im Originalsystem war das gar nicht vorgesehen.
- **Durchgaengige Plattformweichen:** Fast jede Datei-, Ordner- und E-Mail-Operation braucht eigenen Code fuer Mac (#If Mac, AppleScript) und Windows (FileDialog, Outlook-COM) - jede Aenderung musste zweimal gepflegt werden.
- **Gmail wurde nie wirklich unterstuetzt** - der entsprechende Code-Pfad oeffnet im Zweifel nur ein leeres Programmfenster und gibt sonst nichts zurueck.
- **Unvollstaendiger Code an zentraler Stelle:** Die Validierung der Dashboard-Suche (H1_Suchen_Funktionen) enthielt noch Platzhalter- und auskommentierten Code fuer eine nie fertiggestellte Zusatzfunktion.
- **Sicherheitsschwache Absicherung:** Der Blattschutz nutzte ein kurzes, im Klartext im Quellcode hinterlegtes Passwort.
- **Fehlende Nachvollziehbarkeit von Loeschvorgaengen:** Ein geloeschtes Artikel-Stueck (Del_ArtikelStueck) war sofort und ohne jede Wiederherstellungsmoeglichkeit weg.

Diese Beobachtungen flossen direkt in die Anforderungen an die Neuentwicklung ein: Mehrbenutzerfaehigkeit von Anfang an, eine einzige Codebasis ohne Plattformweichen, eine bewusst einfach gehaltene E-Mail-Loesung (mailto-Links statt Client-Automatisierung) sowie - spaeter im Projekt ergaenzt - eine Papierkorb-Funktion mit Wiederherstellungsmoeglichkeit.

Teil II - Die Neuentwicklung als Web-Anwendung

7. Architekturentscheidung: Weg von Excel

Nach der vollstaendigen Analyse des Originalsystems wurde ueber Alternativen diskutiert. Nach Abwaegung verschiedener Optionen fiel die bewusste Entscheidung, **nicht** bei Excel zu bleiben, sondern das System als eigenstaendige Web-Anwendung mit einer robusteren Datenbank im Hintergrund neu aufzubauen - unter der ausdruecklichen Vorgabe, die Loesung **so einfach wie moeglich** zu halten und das Projekt **Schritt fuer Schritt** gemeinsam zu entwickeln. Als Zielarchitektur wurde eine Kombination aus **Python**, dem Web-App-Framework **Streamlit** und einer **SQLite**-Datenbank gewaehlt: Streamlit erzeugt aus reinem Python-Code eine vollstaendige, plattformunabhaengige Web-Oberflaeche - kein Frontend-Framework, kein manuelles HTML/CSS/JavaScript, keine Mac/Windows-Verzweigungen mehr noetig. SQLite speichert die gesamte Datenbank in einer einzigen Datei, laeuft aber - anders als Excel - mehrbenutzerfaehig im WAL-Modus.

Von Beginn an wurde die Anwendung in drei klar getrennte Schichten aufgeteilt, die sich durch das gesamte Projekt hindurch bewaehrt haben: **app.py** (Streamlit-Oberflaeche: alle Seiten, Formulare, Dialoge, Navigation), **logic.py** (fachliche Geschäftslogik: Barcode-Vergabe, Materialentnahme-Logik, Suche, Export) und **db.py** (Datenzugriffsschicht, kapselt saemtliche SQL-Zugriffe hinter generischen Funktionen) - eine deutlich klarere Trennung als im VBA-System, wo UserForm-Code, Fachlogik und Tabellenzugriff oft im selben Modul vermischt waren.

8. Aufbau des Grundsystems

8.1 Datenmodell

Das neue Datenbankschema wurde bewusst nah an der Struktur der Excel-Tabellen gehalten (siehe Kapitel 5), um die fachliche Logik direkt uebertragen zu koennen: **Lager**, **Typ**, **Einheit**, **Profil_Abk** und **Material_Gruppe** als Stammdaten, **Artikel_Gruppe** als Artikel-Stammsatz, sowie die drei identisch aufgebauten Tabellen **Artikel_Liste**, **Reste** und **Verlauf**. Das Barcode-Schema (Haupt-Barcode ohne Bindestrich, Stueck-Barcode mit „-Nummer“) wurde unveraendert uebernommen.

8.2 Materialentnahme-Logik

Die Kernlogik aus `G7_MaterialEntnahme_Logik` wurde nahezu 1:1 in `logic.process_material_entnahme()` uebertragen: Verlauf-Eintrag vor der Reduktion, Reduktion/Loeschung des Ursprungsstuecks, anschliessend bedingtes Einbuchen des Reststuecks (mit Zusammenfuehrung bei identischer Laenge). Neu hinzugekommen ist die spaeter eingefuehrte, ueber die Oberflaeche **einstellbare** Schrott-/Rest-Schwelle (im Original war „1000 mm“ fest im Code verankert).

8.3 Mehrbenutzerbetrieb

Da mehrere Personen im selben Netzwerk gleichzeitig zugreifen sollten, wurde die SQLite-Datenbank im **WAL-Modus** betrieben, wodurch sich lesende und schreibende Zugriffe nicht gegenseitig blockieren. Die Vergabe neuer Barcodes wurde zusaetzlich durch einen anwendungsinternen Lock abgesichert. Die Robustheit wurde mit einem Belastungstest von 100 gleichzeitigen Schreibzugriffen (5 parallele simulierte Nutzer) verifiziert - ohne Konflikte oder doppelte Barcodes. Dieser gesamte Themenbereich existierte im Original ueberhaupt nicht.

8.4 Export und Kommunikation

Statt der plattformabhaengigen Ordnerdialoge und Mail-Client-Automatisierung aus den B- und C-Modulen setzt die neue Anwendung auf Streamlits eingebauten Datei-Download (Browser waehlt den Speicherort) sowie auf einen einfachen **mailto**-Link fuer den E-Mail-Entwurf - bewusst ohne SMTP-Zugangsdaten, da fuer aber garantiert auf jeder Plattform und mit jedem Mail-Anbieter (inklusive Gmail) gleich funktionierend.

9. VBA-Funktion vs. Python-Funktion - eine Gegenueberstellung

Die folgende Tabelle zeigt beispielhaft, wie zentrale VBA-Funktionen des Originalsystems in der neuen Anwendung wiederzufinden sind:

VBA (Original)	Python (Neu)	Zweck
FindeMaxBarcodeNummer	logic.make_barcode_ag / _naechste_stueck_nummer	Naechste freie Barcode-Nummer ermitteln
Rowsuchen / WertExistiertInTabelle	db.value_exists / SQL WHERE-Abfragen	Datensatz-Suche/Duplikatpruefung
MakeNew_ArtikelStueck	logic.add_artikel_stueck	Neues Artikel-Stueck einbuchen
ProzessMaterialEntnahme	logic.process_material_e ntnahme	Materialentnahme mit Schrott-/Rest-Logik
Del_ArtikelStueck	logic.soft_delete_artike l_stueck + Papierkorb	Loeschen (im Original endgueltig, jetzt wiederherstellbar)
edit_ArtikelStueck	logic.edit_artikel_stuec k	Bestehendes Stueck bearbeiten
MakeArtikelGruppe	logic.create_artikel_gru ppe	Neue Artikelgruppe anlegen
DashboardExport / B3_Exportieren	logic.export_excel_bytes / export_pdf_bytes	Excel-/PDF-Export
VerschickeExportPerEmail	logic.hole_email_betreff /-text + mailto-Link	E-Mail-Entwurf
BtnSuche_Click / Fill_Dashboard_List	logic.suche / erweiterte_suche	Dashboard-Suche
SheetsSperren / SheetsEntsperren	entfaellt (kein Blattschutz-Konzept mehr noetig)	Zugriffsschutz

10. Digitalisierung der Bestandsdaten

Die realen Lagerbestaende lagen zunaechst nur als handschriftliche Notizen vor (vier Fotos handbeschriebener Listen). Um Fehler bei der Uebertragung zu vermeiden, wurde zunaechst **keine** direkte Uebernahme in die Produktivdatenbank vorgenommen. Stattdessen wurde aus den Fotos eine Kontroll-Tabelle erstellt, die alle erkannten Werte samt unsicherer Stellen auflistete. Diese wurde vom Auftraggeber vollstaendig durchgesehen und korrigiert. Erst auf Basis der final bestaetigten Tabelle wurden die Daten importiert: **86 Artikel-Datensaetze** ueber 50 Artikelgruppen hinweg - die von diesem Zeitpunkt an durchgehend die reale Produktivgrundlage bildeten, gegen die jede weitere Aenderung getestet wurde.

11. Iterative Weiterentwicklung der Oberflaeche

Im Anschluss an das Grundsystem folgte eine laengere Phase kleinteiliger, nutzerzentrierter Verbesserungen - jede einzeln umgesetzt, im Browser getestet und erst danach die naechste angegangen:

- PDF-Download zusaetzlich zum eingebauten CSV-Download bei jeder Tabellenansicht, spaeter bei gefilterten/gesuchten Tabellen einheitlich oben neben der Ergebnis-Ueberschrift platziert.
- Die haeufigsten Aktionen (Hinzufuegen, Material entnehmen, Bearbeiten/Loeschen, Artikelgruppe anlegen) von eigenen Unterseiten auf modale Popup-Fenster umgestellt: vertikal zentriert, Felder untereinander statt nebeneinander, Bestaetigungs-Buttons konsequent rechts.
- Die sechs urspruenglich getrennten Buttons fuer „Artikel“ und „Reste“ zu drei gemeinsamen Buttons zusammengefuehrt - die Laenge entscheidet seither automatisch ueber Voll/Rest.
- Tabellen zeigen ihre volle Hoehe ohne inneren Scrollbalken.
- Die Textsuche zu einer tipp- und scanfaehigen Dropdown-Kombobox ausgebaut, ergaenzt um einen eigenen Abschnitt „Erweiterte Filter“ mit Mehrfachkriterien.
- Die Schrott-/Rest-Laengengrenze (im Original fest im Code) ueber eine Konfigurationstabelle einstellbar gemacht und beide Schwellenwerte auf eine gemeinsame Einstellung vereinheitlicht.
- Schnellzugriff-Buttons zentral in die Seitenleiste verschoben, damit sie von jeder Seite aus verfuegbar sind.
- Zahlreiche weitere Feinjustierungen: Ueberschriftengroessen, Spaltenanzahl in Formularen, einheitliche Button-Breiten, eigene Filterleisten direkt auf „Artikel Liste“ und „Reste“.

12. Datensicherheit: Papierkorb und Loesch-Bestaetigung

Im Original loeschte `Del_ArtikelStueck` einen Artikel nach einer einzigen Ja/Nein-Abfrage sofort und endgueltig. Im Zuge der Vorbereitung auf den produktiven Einsatz wurde dieses Risiko erneut aufgegriffen und diesmal vollstaendig entschaeft: Vor jedem Loeschvorgang erscheint eine explizite Sicherheitsabfrage, und ein geloeschtes Stueck landet nicht mehr sofort im Nichts, sondern in einer neuen **Papierkorb**-Tabelle samt Loeschzeitpunkt - von dort entweder wiederherstellbar oder, nach einer zweiten eigenen Sicherheitsabfrage, endgueltig loeschbar.

Teil III - Von lokal zur Cloud

13. Produktionsreife auf dem lokalen Rechner

13.1 Automatisches Backup

Ein Backup-Skript sichert die Datenbank ueber SQLites **VACUUM INTO**-Befehl in eine eigenstaendige, konsistente Kopie - anfangs nur beim Start der Anwendung ausgeloeset, spaeter um einen zusaetzlichen, taeglich zu fester Uhrzeit laufenden Zeitplan ergaenzt, unabhaengig von Neustarts. Es werden automatisch die letzten 30 Tage aufbewahrt.

13.2 Automatischer Hintergrunddienst

Ein macOS-**LaunchAgent** sorgt dafuer, dass die Anwendung automatisch beim Anmelden am Rechner startet und sich bei einem Absturz selbststaendig neu startet - verifiziert durch gezieltes Beenden des Prozesses im laufenden Betrieb.

13.3 Umzug aus iCloud Drive

Der Projektordner lag urspruenglich in iCloud Drive; macOS verweigert Hintergrunddiensten dort systembedingt den Dateizugriff. Das gesamte Projekt wurde daraufhin in einen lokalen, nicht synchronisierten Ordner verschoben (die urspruengliche Kopie blieb - nur umbenannt - als Sicherheitsnetz erhalten), die virtuelle Python-Umgebung neu aufgebaut und der Hintergrunddienst entsprechend umkonfiguriert.

13.4 Kurzanleitung fuer den Vorgesetzten

Ergaenzend zur technischen README wurde eine kurze, nicht-technische Anleitung fuer die taegliche Nutzung erstellt: Oeffnen der App, Uebersicht der Seiten, wichtigste Aktionen im Schnellzugriff, Vorgehen bei versehentlichem Loeschen.

14. Mehrgeraete-Zugriff und Windows-Vorbereitung

Fuer den Zugriff weiterer Mitarbeitender wurden fertige Verknuepfungen erstellt (macOS „.webloc“, Windows „.url“, jeweils direkt auf die Netzwerkadresse zeigend, kein Installationsschritt noetig), sowie fuer iOS/Android die Vorgehensweise „Zum Home-Bildschirm hinzufuegen“ dokumentiert.

Da die urspruengliche Loesung an den privaten Rechner des Praktikanten gebunden war, wurde zusaetzlich ein vollstaendiges **Windows-Setup-Paket** vorbereitet: ein plattformunabhaengiges Backup-Skript (Python statt Bash), ein Start-Skript fuer den Hintergrundbetrieb ueber die Windows-Aufgabenplanung (unsichtbar per VBS-Wrapper gestartet), ein manueller Doppelklick-Fallback sowie eine ausfuehrliche Schritt-fuer-Schritt-Anleitung.

15. Die Entscheidung fuer die Cloud

Bei der Planung der Uebergabe wurde ein struktureller Schwachpunkt erkannt: Der Praktikant, auf dessen Rechner die Anwendung lief, wurde das Unternehmen irgendwann verlassen - ein System, das an eine einzelne Person gebunden ist, waere dann nicht mehr betriebsfaehig. Zwei Wege wurden gegenuebergestellt: die Verlagerung auf einen Firmenrechner (kostenlos, aber weiterhin auf das Buero-Netzwerk beschraenkt) oder echtes Cloud-Hosting (von ueberall erreichbar, aber mit echten Kosten, notwendigem Zugriffsschutz und einem groesseren technischen Umbau, da eine einzelne SQLite-Datei nicht zu den meisten Cloud-Plattformen passt). Die Entscheidung fiel zugunsten des Cloud-Wegs: **GitHub** fuer die Quellcodeverwaltung, **Neon** als kostenlose gehostete Postgres-Datenbank und **Streamlit Community Cloud** als Ausfuehrungsumgebung mit privatem, einladungsbasiertem Zugriff.

16. Technische Umsetzung der Cloud-Migration

16.1 Zwei-Wege-Datenzugriffsschicht

Statt die lokale SQLite-Loesung zu ersetzen, wurde **db.py** so erweitert, dass sie beide Betriebsarten beherrscht: Ist eine **DATABASE_URL** konfiguriert, verbindet sich die Anwendung mit Postgres, andernfalls bleibt sie beim lokalen SQLite-Betrieb - exakt derselbe Anwendungscode laeuft wahlweise lokal oder in der Cloud, ohne Verzweigungen (eine bewusste Abkehr von der Mac/Windows-Verzweigungslogik des Originalsystems, siehe Kapitel 6).

16.2 Herausforderungen bei der Umstellung

- **Gross-/Kleinschreibung von Bezeichnern:** Postgres faltet unquotierte Tabellen-/Spaltennamen automatisch auf Kleinschreibung. Ein eigenes Zeilen-Objekt sorgt seither dafuer, dass Ergebniszeilen die im Schema definierte Schreibweise zeigen - sowohl bei direktem Feldzugriff als auch nach Umwandlung in ein Python-Dictionary.
- **Cursor-Typ-Konflikt mit pandas:** behoben durch getrennte Cursor-Konfigurationen.
- **Alias-Kollision** zwischen einem Abfrage-Kurznamen und einer echten Spalte gleichen Namens.
- **Sequenz-Fortsetzung:** nach dem Uebertragen bestehender Datensaeetze musste der interne Postgres-Zaehler manuell auf den hoechsten uebertragenen Wert gesetzt werden.

Jede Korrektur wurde lokal gegen eine eigens installierte Postgres-Testinstanz mit einer Kopie der echten 86 Datensaeetze nachgestellt und durch einen vollstaendigen Klick-Test aller Seiten und Dialoge verifiziert, bevor sie live geschaltet wurde.

17. Go-Live: GitHub, Neon und Streamlit Community Cloud

- Anlegen eines **privaten** GitHub-Repositorys (lokale Datenbank, Backups und Zugangsdaten ueber .gitignore bewusst ausgeschlossen).
- Einrichten einer kostenlosen **Neon**-Postgres-Datenbank, Uebertragung der 86 echten Artikel-Datensaeetze samt aller Stammdaten mittels eines eigens gebauten Migrationsskripts.
- Deployment auf **Streamlit Community Cloud**, Datenbankverbindung als geschuetztes Secret hinterlegt.
- Einschraenkung der Sichtbarkeit auf ausgewaehlte, eingeladene Personen.

Nach dem Deployment wurde die Live-Anwendung direkt im Browser gegen die echten Daten getestet - bei laufender Verifikation, dass der parallel weiterlaufende lokale Mac-Betrieb davon unberuehrt blieb.

18. Fehlerbehebung nach dem Go-Live

Kurz nach der ersten Veröffentlichung meldete der Auftraggeber Fehler beim Aufrufen der Seiten „Artikel Liste“/„Reste“ sowie beim Anlegen eines neuen Artikels. Beide Fehler liessen sich auf dieselbe Grundursache zurueckfuehren - Postgres-spezifische Eigenheiten bei der Gross-/Kleinschreibung - traten aber an Stellen auf, die von den urspruenglichen Tests nicht abgedeckt waren. Beide wurden durch eine grundlegend robustere Loesung behoben (das interne Zeilen-Objekt benennt Spalten bereits beim Erzeugen der Zeile korrekt um), erneut vollstaendig lokal nachgestellt, vor der Veroeffentlichung durch einen kompletten Klick-Test bestaetigt und anschliessend live verifiziert.

19. Aktueller Stand und offene Punkte

19.1 Was heute funktioniert

- Vollstaendige Abbildung aller Funktionen des Originalsystems - jetzt plattformunabhaengig, mehrbenutzerfaehig und ohne Code-Duplizierung fuer Mac/Windows.
- Papierkorb-Funktion und Loesch-Sicherheitsabfrage als Schutz vor Bedienfehlern (eine direkte Antwort auf die in Kapitel 6 identifizierte Schwachstelle).
- Lokaler Betrieb auf dem Mac laeuft unveraendert als Hintergrunddienst mit automatischem taeglichem Backup.
- Cloud-Version unter einer privaten, einladungs-basierten Adresse erreichbar - unabhaengig von jedem einzelnen Geraet oder jeder einzelnen Person, mit denselben 86 realen Artikel-Datensaetzen.
- Fertig vorbereitetes Windows-Setup-Paket als Alternative bzw. Rueckfalloption.

19.2 Bekannte Einschränkungen

- Die kostenlose Neon-Datenbank pausiert nach ca. 5 Minuten Inaktivitaet und braucht beim naechsten Zugriff einen kurzen Moment zum Aufwachen.
- Automatische Datensicherung erfolgt in der Cloud-Variante durch Neon selbst (Point-in-Time-Recovery, im kostenlosen Plan fuer die letzten 6 Stunden).
- Die Netzwerkadresse des lokalen Mac-Betriebs ist nicht als feste IP im Router hinterlegt.

Insgesamt wurde das Projekt von der vollstaendigen Analyse eines gewachsenen, plattformabhaengigen und an mehreren Stellen unfertigen Excel/VBA-Systems bis zu einer eigenstaendigen, mehrfach abgesicherten und privat erreichbaren Cloud-Anwendung gefuehrt - mit durchgaengigem Fokus auf Einfachheit fuer die Endnutzer, schrittweiser Umsetzung sowie sorgfaeltiger Absicherung der echten Produktivdaten bei jeder einzelnen Aenderung.